

Code Konzept

Digitales OP-Planungs- & Ressourcenmanagement

Fortgeschrittene Programmierung

Katja Fichtner-Pflaum

Kernfunktionen

Das System ist ein dynamisches OP-Planungssystem für eine Tagesklinik. Das System definiert einen OP-Plan in Abhängigkeit von vorhandenen Ressourcen. Es werden die Belegungen der OP-Säle dargestellt und zeigt an, wie viel Restzeit in einem Saal zur Verfügung steht.

Das System soll folgende Kernfunktionen bereitstellen:

- Das System kennt das im Haus vorhandene Personal, die Geräte, die Instrumente (inklusive deren Wiederverfügbarkeit nach dem Sterilisationsprozess) sowie den Bestand an Einmal- und Verbrauchsmaterialien (Fäden, Pflaster etc.).
- Ressourcen der verschiedenen OP-Arten: Ein integriertes Array, in dem gespeichert ist, was für Ressourcen für die entsprechende Operation benötigt wird. Hier wird definiert, wie viel Personal, welche Geräte, welche Instrumente und Einmalartikel für eine OP-Art benötigt werden.
- Jeder OP-Saal besitzt ein definiertes Zeitfenster pro Tag (z. B. 08:00 bis 16:00 Uhr). Das System zeigt an, wie viele Minuten in welchem Saal noch frei sind.
- Es gibt die Möglichkeit über den OP-Manager die laufende OP zu verkürzen oder zu verlängern. Folgende OPs werden darauf hin nach vorne/hinten aktualisiert und die Wiederverfügbarkeit von Personal und Sterilgut wird aktualisiert.
- Wenn die OP-Dauer kürzer als erwartet gespeichert wird, wird berechnet, welche neuen OPs in die gewonnene Zeit eingepflegt werden könnten.

Klassenstruktur

Attribute und Methoden von Klassen

Klasse: Ressource

Repräsentiert alle Entitäten im Haus, die für eine OP blockiert werden müssen (Geräte, Personal, Einmalartikel, Instrumente)

Attribut	Typ	Beschreibung
name	str	Eindeutige Kennung („Operator“, „Röntgengerät“, „Vicryl3-0“)
verfuegbar	bool	Gibt an, ob Ressource verfügbar und einsatzbereit ist.
eingesetzt_in_op	str	OP-Name, in der die Ressource eingesetzt ist

Methode	Beschreibung
ist_verfuegbar(von_minute: int, bis_minute: int) ->bool	Prüft, ob Ressource in gewissem Zeitfenster zeit hat
blockieren(op_name: str)	Reserviert Ressource für eine bestimmte OP
freigeben()	Ressource wird wieder für das System verfügbar gemacht

Klasse Instrument – erbt von Ressource

Attribut	Typ	Beschreibung
steri_bis_minute	int	Bis zu Minute x ist Instrument in Sterilisationsprozess.

Methode - Instrumente	Beschreibung
starte_sterilisation(end_minute_op: int)	Setzt den Status verfügbar erst nach Sterilisationsprozess auf True
pruefe_einsatzzeit(ziel_minute: int) ->bool	Prüft, ob Instrumente zu Zeitpunkt x verfügbar sind

Klasse Einmalartikel - erbt von Ressource

Attribut	Typ	Beschreibung
meldebestand	int	Schwellenwert. Wird dieser unterschritten, wird ein Nachbestell-Trigger ausgelöst.
bestand	int	Aktueller Lagerbestand

Methode	Beschreibung
konsumiere(menge: int)	Prüft Lagerbestand der Einmalartikel. Wenn $\text{bestand} \leq \text{meldebestand}$ wird aufgerufen den Artikel nachzubestellen.

Klasse OP Saal

Wie viele OP Säle sind in unsere Klinik verfügbar und wie lang können die Säle entsprechend belegt werden.

Attribut	Typ	Beschreibung
saal_id	str	Eindeutige Kennung des Saals („Saal_01“)
kapazitaet_minute	int	Gesamte tägliche Betriebszeit in Minuten
geplante_ops	list[OP]	Zeitlich sortierte Liste aller Operationen, die in diesem Saal gebucht sind.
reinigung	int	Reinigungszeit nach jeder OP in diesem Saal

Methoden	Beschreibung
berechne_restzeit() ->int	Berechnet die noch verfügbaren Freiminuten des Saals
op_hinzufuegen (neue_op: OP)	Prüft ob neue OP zeitlich, ohne Überschneidung, in den Saal passt
aktualisiere_zeitstrahl (ab_minute: int, verschiebung: int)	Schiebt alle nachfolgenden OPs um x Minuten nach vorne/hinten (Prüfung Saalkapazität, Ressourcen Instrumente – Fehlerausgabe falls nicht möglich)

Klasse: OPTyp

Definiert verschiedene Operationen mit benötigten Ressourcen und legt deren Zeitfaktor fest.

Attribut	Typ	Beschreibung
op_name	str	Name der OP-Art („Knie TEP“, „Acetabulumfraktur“)
standard_dauer	int	Standard Zeit für jeweiligen Eingriff.
benoetigte_ressourcen	dict[str,int]	Liste der benötigten Ressourcen (Personal, Geräte, Siebe)

Klasse: OP

Zeitlich definierte OP auf dem Zeitstrahl.

Attribut	Typ	Beschreibung
op_name	str	Verweis auf zugrundeliegenden OPTyp
saal_id	str	Nummer des zugewiesenen OP-Saals
start_minute	int	Startzeitpunkt auf dem Zeitstrahl
end_minute	int	Berechnet Endzeitpunkt auf dem Zeitstrahl
geblockte_ressourcen	list[Ressource]	Für den Zeitraum fest zugewiesene Ressourcen

Klasse: OPManager

Das Hauptsystem. Hier werden die Ressourcenpools gesteuert, die Zeitstrahl berechnet und die Saalkapazitäten berechnet.

Attribut	Typ	Beschreibung
ressourcen_pool	list[Ressource]	Gesamtes vorhandenes Personal & Geräte
instrumenten_pool	list[Instrument]	Alle vorhandenen chirurgischen Siebe
material_lager	list[Einmalartikel]	Gesamtbestand aller Einmalartikel
op_katalog	list[OPTyp]	Alle theoretisch möglichen OP-Arten.
op_saele	list[OPSaal]	Liste aller verfügbaren Operationssäle.

Methoden	Beschreibung
fuege_ressource_hinzu (neue_ressource: Ressource)	Fügt dem ressourcen_pool Personen/Geräte/Einmalartikel hinzu
fuege_instrument_hinzu(neues_instrument: Instrument)	Fügt dem instrumenten_pool chirurgische Siebe hinzu
fuege_saal_hinzu(neuer_saal: OPSaal)	Registriert neuen OP-Saal im System
lager_material_ein(name:str, menge:int)	Erhöht Bestand eines existierenden Artikel bei Lieferung
registriere_optyp	Definiert welche Eingriffe die Klinik anbietet und was jeweils für Ressourcen benötigt werden.
zeige_verfuegbare_ressourcen(minute: int)	Scannt alle Pools und gibt aus, welches Personal, welche Geräte und welche Instrumente zu dieser Minute frei sind.
zeige_ops_von_bis(start: int, ende: int)	Filtert den Zeitstrahl und zeigt alle OPs an, die in diesem Zeitfenster liegen.
zeige_aktuelle_ops()	Zeigt OPs die zu aktuellem Zeitpunkt laufen.
op_starten(op_name: str, saal: str, start_minute: int)	Holt OPTyp, prüft Ressourcen, blockiert Ressourcen im Pool und fügt die OP in op_am_laufen ein.
op_zeit_anpassen(op_name: str, neue_dauer: int)	Passt Dauer an und verschiebt Folgetermine mit Methode aktualisiere_zeitstrahl

Beschreibung des Use-Cases & Objektinteraktionen

Die Klasse OPManager sgiert als oberste Steuereinheit des Systems. Sie beseitzt und verwaltet alle Datenpools. Die anderen Klassen (Ressourcen, OPSaal, OP, OPTyp) agieren als Datenentitäten mit eigener Kompetenz.

Die Interaktion der Objekte lässt sich in vier logische Phasen des Klinikalltags unterteilen:

Phase 1: System-Initialisierung und Stammdaten-Setup

Bevor der erste Patient geplant werden kann, baut der OPManager die digitale Klinik auf.

1. Ressourcen- und Materialerfassung: Der OPManager wird instanziiert. Über die Methoden fuege_ressource_hinzu(), fuege_instrument_hinzu() und lager_material_ein() werden die realen Klinikstrukturen abgebildet. Dabei

entstehen eigenständige Objekte im ressourcen_pool (z. B. Personal-Objekte, Geräte-Objekte), im instrumenten_pool (chirurgische Siebe) und im material_lager (Objekte der Klasse Einmalartikel).

2. Katalogerstellung: Über registriere_op_typ() werden OPTyp-Objekte (wie z. B. Knie-TEP) im op_katalog hinterlegt. Jedes OPTyp-Objekt hält in seinem Attribut benoetigte_ressourcen ein Dictionary, das die Anforderungen für diesen Eingriff definiert.
3. Saal-Aktivierung: Über fuege_saal_hinzu() werden OPSaal-Objekte registriert, die jeweils ihre eigene maximale Tageskapazität (kapazitaet_minute) verwalten.

Phase 2: Der reguläre Buchungsprozess (op_starten)

Ein neuer Eingriff soll für den Tag eingeplant werden.

1. Anforderung & Validierung: Der OPManager erhält den Aufruf op_starten(op_name, op_typ_name, saal_id, start_minute). Er sucht das entsprechende OPTyp-Objekt aus dem op_katalog.
2. Ressourcenprüfung: Der OPManager gleicht die im OPTyp hinterlegten Anforderungen mit dem ressourcen_pool und dem instrumenten_pool ab. Er iteriert durch die Ressourcen-Objekte und ruft deren Methode ist_verfuegbar(von_minute, bis_minute) bzw. pruefe_einsatzzeit() auf.
3. Materialverbrauch: Für benötigte Verbrauchsmaterialien steuert der Manager das material_lager an. Das betroffene Einmalartikel-Objekt führt die Methode konsumiere(menge) aus. Unterschreitet das Attribut bestand nun den meldebestand, löst das Objekt intern den Nachbestell-Trigger aus.
4. Instanziierung der OP: Sind alle Ressourcen frei und genug Materialien da, erzeugt der OPManager ein konkretes Objekt der Klasse OP. Dieses Objekt speichert den Zeitraum (start_minute, end_minute) und erhält im Attribut geblockte_ressourcen Referenzen zu den exakten Ressource-Objekten (z. B. "Operateur", "Sieb_05").
5. Einplanung im Saal: Das neue OP-Objekt wird an das Ziel-OPSaal-Objekt übergeben. Die Methode op_hinzufuegen() prüft, ob der Zeitraum im Saal kollisionsfrei frei ist. Wenn ja, wird die OP in die Liste geplante_ops einsortiert. Das Saal-Objekt aktualisiert über berechne_restzeit() seine verfügbaren Freiminuten.

Phase 3: Live-Monitoring im laufenden Betrieb

Während des Tages liefert das System dem OP-Koordinator den aktuellen Status.

1. Verfügbarkeits-Abfrage: Wird `zeige_verfuegbare_ressourcen(minute)` aufgerufen, scannt der OPManager alle Pools. Er fragt bei den Instrument-Objekten das Attribut `steri_bis_minute` ab. Ist die angeforderte Minute kleiner als dieser Wert, meldet das Objekt `verfuegbar = False`.
2. Saal-Überwachung: Der Manager liest die `op_saele` aus. Jedes OPSaal-Objekt berechnet über `berechne_restzeit()` live, wie viel Pufferzeit (Kapazität abzüglich Summe der Dauern aller geplante_ops inkl. reinigung) noch existiert, um Verspätungen abzufangen.

Phase 4: Dynamische Verschiebung (op_zeit_anpassen)

Eine laufende OP dauert unerwartet 30 Minuten länger (Komplikation).

1. Signal an den Manager: Der Koordinator ruft am OPManager die Methode `op_zeit_anpassen(op_name, neue_dauer)` auf.
2. Anpassung der betroffenen OP: Der Manager sucht das betroffene OP-Objekt. Dieses berechnet sein Attribut `end_minute` neu.
3. Kettenreaktion im Saal: Der OPManager delegiert die Verschiebung an das zuständige OPSaal-Objekt via `aktualisiere_zeitstrahl(ab_minute, verschiebung)`.
4. Iterative Konfliktprüfung: Das OPSaal-Objekt verschiebt alle nachfolgenden OP-Objekte in seiner Liste `geplante_ops` auf dem Zeitstrahl nach hinten.
 - Jede verschobene OP triggert nun erneut die Methoden `ist_verfuegbar()` ihrer in `geblockte_ressourcen` hinterlegten Personal- und Geräte-Objekte für den neuen Zeitraum.
 - Für enthaltene Instrument-Objekte wird `starte_sterilisation(neue_end_minute)` aufgerufen. Das Instrument errechnet das Attribut `steri_bis_minute` neu. Die nächste OP, die dieses Instrument benötigt, prüft über `pruefe_einsatzzeit()`, ob das Sieb bis zu ihrem neuen Startzeitpunkt wieder steril ist.
5. Ergebnis: Gibt es bei einer der Folge-OPs einen Ressourcenkonflikt (z. B. Instrument ist noch in der Sterilisation) oder wird die `kapazitaet_minute` des Saals überschritten, bricht die Methode ab und wirft eine Fehlermeldung (Exception) aus, damit der OP-Manager manuell umplanen kann. Bei einer Verkürzung berechnet der Manager analog freigewordene Lücken für den Katalog.

Aufwandschätzung (Entwicklungszeit)

Entwicklungsphase + Beschreibung	Geschätzter Aufwand
1. Systemarchitektur & Daten-Setup Erstellen der JSON- oder Dictionary-Strukturen für den initialen Klinik-Katalog (Anlegen von Test-Ärzten, Sälen, Sieben und OP-Typen).	ca. 1,5h
2. Basis-Klassen & Vererbung Implementierung der Klassen Ressource, Instrument und Einmalartikel via Vererbung inkl. der __init__-Konstruktoren und Basis-Prüfungen (ist_verfügbar).	ca. 2,5h
3. OP-Saal & Manager-Grundgerüst Programmierung der Setup-Methoden im OPManager und der Logik zur Restzeitberechnung in OPSaal.	ca. 3,0h
4. Logistik & Material-Trigger Einbau der konsumiere-Logik bei den Einmalartikeln inklusive des Meldebestand-Abgleichs und Fehlermeldungen (Exceptions), falls Material fehlt.	ca. 2,0h
5. Kern-Algorithmus (Zeitschieber) Dynamisches Vor- und Zurückrücken der Folgetermine im Saal. Einbau der Kettenreaktion (prüfen, ob nachfolgende OPs durch die Verschiebung mit Arzt-Schichten kollidieren oder Instrumente noch in der Steri sind).	ca. 9,0 Std.
6. Simulation & Konsolen-Interface Bau eines interaktiven Test-Szenarios (z.B. Zeitstrahl um 08:00 Uhr ausgeben, OP verkürzen, aktualisierten Zeitstrahl ausgeben)	ca. 3,0 Std.
7. Code-Review & Edge-Cases Abfangen von Grenzfällen (z.B. Was passiert, wenn eine Verlängerung die Saal-Schließzeit überschreitet? Was passiert bei Tippfehlern im OP-Namen?).	ca. 3,0 Std.

Effiziente Implementierung & GitHub-Workflow

Architektur für einen effizienten GitHub-Workflow Das Konzept ist stark modular und nach dem Controller-Pattern (OPManager steuert die Entitäten) aufgebaut. Dies würde theoretisch eine parallele Entwicklung im Team via GitHub ermöglichen.

- Feature-Branch-Workflow: Da die Datenhaltung (Kataloge, Klassen) streng von der Algorithmus-Logik (Zeitschieber) getrennt ist, könnten Entwickler zeitgleich an verschiedenen Klassen arbeiten, ohne Merge-Konflikte zu riskieren.
- Durch die Aufteilung in 7 Entwicklungsphasen entspricht jede Phase einem Ziel, das in GitHub dokumentiert wird. Dadurch bleibt eine klare und nachvollziehbare Übersicht erhalten
- Erleichtertes Testing (CI/CD): Da Methoden wie `berechne_restzeit()` pure Daten verarbeiten und keine Abhängigkeiten haben, lassen sich für jede Klasse isolierte Tests schreiben, die automatisiert über GitHub Actions validiert werden können.

Umsetzung der Coding-Guidelines

- Code-Stil: Das Projekt folgt strikt den PEP 8 Richtlinien für Python (z. B. `snake_case` für Funktionen/Attribute, `PascalCase` für Klassennamen).
- Zur Erhöhung der Code-Qualität und Vermeidung von Laufzeitfehlern werden durchgehend Type Hints verwendet (z. B. `von_minute: int`).
- Dokumentation: Jede Klasse und Methode wird mittels Docstrings dokumentiert, um den Code nachvollziehbar zu machen.
- Fehlerbehandlung durch spezifische Custom Exceptions (z. B. `RessourcenKonfliktError`), statt generischer Fehlerausgaben.

Erklärung zum Einsatz von KI-Werkzeugen

Für die Strukturierung und Plausibilisierung der Aufwandschätzung (Entwicklungszeit) im vorliegenden Code-Konzept wurde das KI-basierte Sprachmodell *Gemini* als unterstützendes Werkzeug eingesetzt.

Da es sich hierbei um ein Projekt in einer für mich neuen Komplexitätsstufe handelt und mir in diesem Umfang noch die Erfahrungswerte für eine fundierte Zeiteinschätzung fehlen, wurde die KI gezielt für folgende Aufgaben genutzt:

- Unterstützung bei der systematischen Aufteilung des Projekts in logische, aufeinander aufbauende Entwicklungsschritte.
- Zeiteinschätzung im Bezug auf die einzelnen Phasen.